

VeriReview

Evidence-based verification of GitHub pull-request review resolution

Project review brief: what the system does, how it works, how to run it, where it stands, and what comes next (including the human annotation phase)

Status	Phases 0–13 complete except Phase 8b (semantic evidence in the verdict)
Headline result	Blind test v2 (147 cases): accuracy 0.65, false acceptance 0.15
Mode	Advisory only: neutral GitHub check; merge blocking built but locked
Stack	Python 3.13 · FastAPI · PostgreSQL 16 · Tree-sitter · Docker · GitHub App
Tests	1,108 unit + 37 integration + 2 model tests, strict mypy, ruff, CI

Contents

1. The problem and the one question VeriReview answers
2. How it works: the verification pipeline
3. A worked example (real output)
4. Deployment pieces: API, GitHub App, enforcement gate, dashboard
5. Technology stack and project layout
6. Current stage: phases, results, honest limitations
7. How to run the whole system (step by step)
8. Future plans
9. The remaining human annotation phase (in detail)
10. Review preparation: likely questions and answers
11. Glossary

How to read the numbers. Every number in this brief comes from the project's recorded evaluation reports (experiments/*.json) and README. "Blind" means the test cases were frozen by hash before the rules were changed and evaluated exactly once.

1. The problem

On GitHub, a reviewer leaves a comment on a pull request ("validate username and return HTTP 400"). The author pushes changes and clicks **Resolve conversation**. Nothing checks that the change actually did what was asked: a thread can be resolved with a partial fix, a fix in the wrong place, or only a comment that mentions the fix.

VeriReview answers one question:

Did the code changes made after the review comment actually satisfy the review requirement?

Its answer is a verdict backed by cited evidence (file, lines, commit):

Verdict	Meaning
SATISFIED	every requirement in the comment is met by the code at the end of the window
PARTIALLY_SATISFIED	some requirements met, some not
NOT_SATISFIED	no requirement met (nothing changed, wrong place, only a comment, wrong value...)
UNCERTAIN	the request is ambiguous, or the rules cannot verify it: goes to a human

A separate **policy** layer turns verdict + confidence into an action: ALLOW / WARN / HUMAN_REVIEW / BLOCK. **BLOCK is disabled:** the system is advisory until it is statistically proven safe per category (Section 4.3).

Design principles

- **Evidence, not opinion.** Every conclusion cites the structural fact it rests on.
- **Deterministic rules decide; models only advise.** A semantic model is one evidence source, never the final authority (Phases 6–7 showed similarity models accept bad fixes).
- **When a rule cannot verify, say UNCERTAIN, never accept.** False acceptance (a bad fix accepted) is the safety-critical metric.
- **Repository content is untrusted data.** Code comments, commit messages and replies are never instructions (prompt-injection tested). Repository code is never executed.
- **Honest evaluation.** Test sets are frozen by hash before rule changes and used once.

2. How it works: the verification pipeline

GitHub PR → ingestion → ReviewCase → requirement extraction → evidence → aggregation → policy
 (REST + thread + before/after comment -> atomic, diff, AST, verdict + ALLOW / WARN / GraphQL) commit window code, diff, categorised requirements rules, tests confidence + HUMAN_REVIEW + flags tests + ambiguity score code model* explanation (BLOCK locked)

* the code model runs only in the optional phase7-semantic pipeline and is neutral.

Stage	What it does	Phase
Ingestion	Rebuilds the review thread from GitHub (REST + GraphQL for resolution state) and the temporal window : the commits made after the comment. Handles rebases / force-pushes and sets reliability flags (e.g. resolution time unknown).	1
ReviewCase	One self-contained record: thread, code before and after, diff, changed tests, flags (Pydantic contracts shared across the system).	2
Diff + syntax	unidiff / difflib diffs; Tree-sitter parses Python into symbols and structural facts (calls, conditions, raises, handlers, returns). Finds the commented function even if it was renamed, moved, or extracted into a helper.	3
Requirements	Rule-based extraction: splits the comment into atomic requirements (one action, one target), assigns a category (naming, validation, testing, error_handling, api_behavior, other), targets, conditions, GitHub suggestion blocks, and an ambiguity score .	4
Rules	One deterministic rule per category that queries code structure: guards, handlers, HTTP responses, test inputs. A comment that only mentions the fix never counts. Suggestion blocks are checked against their exact code; common other requests (remove this, add a docstring, use A instead of B) have rules too.	5, 10.1
Aggregation	Per-requirement status → overall verdict. Ambiguous requests → UNCERTAIN. Confidence is lowered when the case is unreliable. HIGH confidence is never emitted.	5, 8a
Explanations	Human-readable explanation that cites each evidence id with file:lines@commit.	8a
Policy	Verdict × confidence → action, in observe / advisory / human_review / enforcement mode. BLOCK additionally needs an eligible category and all four locks open.	8a, 12
NLP baselines	Keyword overlap, TF-IDF and sentence-embedding (MiniLM) similarity, scored by the same harness, for comparison only.	6
Code model	UniXcoder relates each requirement to the most relevant added code (comments removed). Neutral evidence : it never changes a verdict (ADR-002).	7

Why rules and not a model?

Phases 6–7 measured similarity models directly. They reward a comment that repeats the reviewer's words, and even genuinely relevant code can be wrong (the right check in the wrong place still scored 0.73). At any usable threshold they accepted 12.5–87.5% of bad fixes on held-out data, versus 0% for the rules. So models are kept as informational evidence only.

3. A worked example (real tool output)

Request: "validate username and return HTTP 400". The developer added the validation but no 400 response. Output of verireview verify-fixture dataset/fixtures/api-002-validation-without-400, abridged to 6 of its 19 evidence lines (the tool's □ / □ marks are shown as + / x):

Requirements:

+ R1 (validation): validate `username` (non-empty, max 32 chars)
x R2 (api_behavior): return HTTP 400 on invalid input

Evidence:

- [E3] Commented line 10 is in function `signup`. (web/signup.py:7-11 @ 64d302f (before))
- + [E15] Emptiness check `not username or len(username) > 32` on `username` raises `ValueError`
- + [E16] Range check `not username or len(username) > 32` on `username` raises `ValueError`
- + [E17] R1 (validation) satisfied: `username` is checked before use.
- x [E18] No response with HTTP 400 on the requested branch.
- x [E19] R2 (api_behavior) not_satisfied: HTTP 400 is not returned where requested.

Result: PARTIALLY_SATISFIED Confidence: MEDIUM

Recommended action: HUMAN_REVIEW (advisory mode). Only part of the request appears satisfied.

4. Deployment pieces

4.1 HTTP API (FastAPI)

Endpoint	Purpose
POST /verify	verify a ReviewCase sent in the body
POST /verify/github	fetch a thread from GitHub (repo, PR, comment id) and verify it
POST /github/webhook	GitHub App deliveries; HMAC-SHA256 signature checked on the raw body before parsing (401 if forged, 503 if no secret configured)
GET /dashboard...	read-only operator pages (404 unless a dashboard token is configured)
GET /health, /health/db	liveness and database readiness

4.2 GitHub advisory mode (Phase 11)

Reviewer resolves a thread → GitHub sends signed webhook → POST /github/webhook (HMAC check)
→ job row in PostgreSQL queue (unique delivery id; claimed with SKIP LOCKED; retry + backoff)
→ worker: App JWT → installation token scoped to ONE repo (contents/PR read, checks write)
→ re-reads thread + code from GitHub → pipeline → audit row (inputs sha256, full result)
→ one neutral "VeriReview" Check Run on the PR head (with a "Confirm reviewed" button)

- Redelivered webhooks are processed once (idempotent queue).
- The webhook body is never trusted: only validated ids are taken; everything is re-read from GitHub.
- Secrets (App key, webhook secret) come from settings as SecretStr, never logged.

4.3 Staged enforcement (Phase 12): built but inert

A check can only turn into a failing (blocking) check when **four independent locks** are all open:

#	Lock	Today
1	Global switch VERIREVIEW_POLICY_ALLOW_BLOCK	off (pinned by a test)
2	Per-repository stage: observe → advisory → human_review → enforcement, one step at a time, recorded; enforcement only after ≥ 14 days of human review and ≥ 10 reviewer confirmations	no repository promoted
3	Eligible category: one-sided 95% Clopper–Pearson upper bounds of false acceptance and false blocking both ≤ 5% on a frozen test run (ADR-003)	none eligible
4	The repository itself marks the check as required (branch protection)	repository's choice

Why no category is eligible: with 0 errors in n cases, the 95% upper bound is about $3/n$, so you need at least **59 clean cases** of each kind per category to get under 5%. The v2 test set has 18–28 cases per category; even naming's clean 0/10 has a 26% upper bound.

4.4 Read-only dashboard (Phase 13)

- Server-rendered in FastAPI with Jinja2 templates, **no JavaScript**, strict Content-Security-Policy, Cache-Control: no-store.
- Off by default: enabled only when VERIREVIEW_DASHBOARD_TOKEN is set. Sign-in with that token creates an HMAC session cookie (HttpOnly, SameSite=Strict, 8 h); failed sign-ins are rate-limited (10 per 5 min).
- Pages: overview (verdict and job totals, gate status), repository (stage, confirmations, history), pull request (threads and verdicts), audit detail (thread, requirements, evidence, explanation, code before/after, diff, inputs hash), jobs queue.
- Stored code is private: verireview purge-audit removes it after 90 days (verdict, hashes and evidence stay as the audit trail).

5. Technology stack and project layout

Layer	Technology
Language / tooling	Python 3.13, uv (dependency lock), ruff (lint + format), strict mypy, pytest
Parsing	Tree-sitter (Python grammar, pinned), unidiff / difflib
Service	FastAPI + uvicorn, Pydantic contracts, Jinja2 (dashboard)
Storage	PostgreSQL 16 (Docker, host port 5433), SQLAlchemy, Alembic migrations 0001–0005
GitHub	REST + GraphQL client; GitHub App auth with PyJWT[crypto] (RS256)
Optional NLP group	scikit-learn, sentence-transformers (MiniLM), transformers (UniXcoder), PyTorch CPU; never in the service image (526 MB)
Deployment	Docker Compose: db, api, worker (profile advisory); GitHub Actions CI

src/verireview/ gh/ threads/ ingestion/ contracts/ diff/ syntax/ requirements/ rules/ evidence/ verification/ explanations/ policy/ semantic/ evaluation/ benchmark/ advisory/ enforcement/ dashboard/ api/ db/ dataset/ experiments/ docs/ (roadmap, per-phase docs, ADRs) migrations/ tests/	GitHub client, thread reconstruction, temporal window, ReviewCase shared Pydantic models (ReviewCase, Requirement, Evidence, Result) diffs; Tree-sitter symbols, facts, location resolution requirement extraction (lexicon in one file); one rule per category evidence stages, pipelines, aggregation, ambiguity gate, reliability cited explanations; ALLOW / WARN / HUMAN_REVIEW / BLOCK baselines, UniXcoder, metrics, injection suite splits, mining, pseudonymisation, annotation page, kappa, gold, freeze GitHub App (webhooks, queue, worker, check); stages + eligibility gate read-only pages; HTTP routes; database models
---	--

6. Current stage

6.1 Phase progress

Phase	What	Status
0	Skeleton: uv, FastAPI /health, PostgreSQL + Alembic, Docker, CI	Done
1	GitHub ingestion, temporal window, live check on real PRs (8/10 threads)	Done
2	Contracts, 29 dev fixtures, evaluation harness, locality baseline	Done
3	Diff + Tree-sitter symbols, facts, location resolution	Done
4	Requirement extraction + ambiguity; blind held-out: count 0.844, category F1 0.909	Done
5 / 5.1	Per-category rules; held-out blind 0.625 (FAR 0.000), then hardening	Done
8a	MVP: confidence, policy, explanations, API, end-to-end tests (15/15 criteria)	Done
6	NLP baselines (lexical, TF-IDF, MiniLM) through the same harness	Done
7	UniXcoder as neutral evidence; prompt-injection suite (0 changes in 2,576 variants)	Done
9	Benchmark v1: annotation guide, blind test cases, 50 real-world cases, frozen manifest, annotation tooling	Done
10	Pre-registered blind evaluation + ablation (acc 0.48, FAR 0.24)	Done
10.1	Benchmark v2 frozen first; rule fixes on dev; blind v2 run (acc 0.65, FAR 0.15)	Done
11	GitHub advisory mode (offline-tested; live run pending the owner's App)	Done
12	Staged enforcement: four locks, statistical gate; inert (nothing eligible)	Done
13	Read-only dashboard, token off by default, retention purge	Done
8b	Semantic evidence in the verdict: waits for data that justifies it (ADR-002)	Open
—	Human annotation of real-world data (Section 9)	Open

6.2 Headline results: blind test v2 (147 cases)

60 controlled + 30 adversarial + 57 fresh real-world cases, frozen **before** any rule change and evaluated once with a pre-registered protocol. 95% bootstrap intervals in brackets.

System	Accuracy	Macro-F1	False accept. ↓	False block. ↓	Coverage
A lexical overlap	0.320	0.161	0.019	0.875	1.00
B embeddings (MiniLM)	0.279	0.122	0.000	0.963	1.00
B' UniXcoder, code view	0.565	0.259	0.830	0.075	1.00
L change near comment	0.551	0.190	0.981	0.000	1.00
S AST structure	0.612	0.292	0.792	0.013	1.00
F-old: VeriReview, Phase 10 rules	0.422 [0.34, 0.50]	0.456	0.302 [0.18, 0.43]	0.100	0.55
F: VeriReview, Phase 10.1 rules	0.653 [0.58, 0.73]	0.655	0.151 [0.06, 0.26]	0.037	0.68
F with gold requirements	0.592	0.621	0.170	0.163	0.67

- **Old vs new rules on the same cases** (paired bootstrap): accuracy +0.231 [+0.16, +0.31], false acceptance −0.151 [−0.25, −0.07]. Gains on every source (controlled +0.13, adversarial +0.20, real-world +0.35).
- **Baselines that look accurate accept almost everything** (L, S, B': 79–98% false acceptance). Baselines with low false acceptance reject almost everything (A, B: 88–96% false blocking).
- **Real-world**: 25 of 57 cases decided (was 5), 23 correctly, no false acceptance; mostly via the suggestion-block check.
- **Dev vs test gap shrank**: dev 0.798 vs test 0.653 (in Phase 10 it was 0.4–0.5).

6.3 Other results

Measure	Result
Blind test v1 (Phase 10, 120 cases)	accuracy 0.483 [0.39, 0.57], FAR 0.237 [0.14, 0.35]
Requirement extraction, blind held-out	count exact 0.844, category F1 0.909 (targets 0.80 / 0.85)
Prompt injection (4 texts × 7 sites × every fixture)	0 / 1,288 outcome changes for mvp and for phase7-semantic; the lexical baseline flipped 6 (NOT_SATISFIED → SATISFIED)
Code-model evidence (UniXcoder)	ROC-AUC dev 0.569 / held-out 0.737, neutral by design
Confidence (v2 test)	MEDIUM verdicts right 84% of the time, LOW 36%; HIGH never emitted

6.4 Honest limitations

- **Not accurate enough to gate merges:** 8 bad fixes were still accepted on the v2 test (FAR 0.15). Use as an advisory flag for human review.
- **No independent human labels yet.** Controlled/adversarial cases were written by the rules' author (blind to the changes); real-world labels are provisional Claude labels. Inter-annotator agreement (κ) cannot be measured until humans label.
- **Scope:** about two thirds of real review requests are other (refactor, docs, style). Beyond suggestion blocks and a few request shapes they end as UNCERTAIN, by design.
- **Structural, not semantic:** no control-flow analysis; helpers followed one level, same file only. Python only, one commented file per case.
- **Live GitHub App run not yet done:** everything is tested offline with recorded fixtures.
- **Dashboard** uses one shared operator token; the retention purge must be scheduled by the operator.
- Both the v1 and v2 test splits are now used; any new rule change needs a fresh blind v3 test set.

7. How to run the whole system

All commands run from the project folder (E:\Projects\NLP_proj\VeriReview) in a Windows terminal. If the project venv is activated, either run `verireview ...` directly, or run `deactivate` first and use `python -m uv run ...`

7.1 Prerequisites

- Python 3.13 with **uv** (`python -m pip install uv` if missing).
- Docker Desktop, running.
- Optional: a fine-grained read-only GitHub token (for live ingestion); Node.js (for the webhook relay).

7.2 Install and configure

```
copy .env.example .env          (only the first time; never commit .env)
python -m uv sync                install dependencies from uv.lock
python -m uv run pytest          unit tests (should all pass)
```

Create a dashboard token and put it in `.env` as `VERIREVIEW_DASHBOARD_TOKEN=<value>` (keep it only in `.env`):

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

7.3 Start the service (API + database + dashboard)

```
docker compose up -d --build      PostgreSQL on host port 5433, API on http://localhost:8000
                                   (the api container applies all migrations automatically)
python -m uv run pytest -m integration integration tests against the database
```

- Health: `http://localhost:8000/health` → `{"status":"ok"}`; database: `/health/db`.
- Dashboard: `http://localhost:8000/dashboard/login`, sign in with the token. Real data arrives through the GitHub App worker (7.5). **For a demo**, fill it first with `python -m uv run verireview demo-seed`: real verifications of the 29 dev fixtures in three fictional demo/... repositories (installation 0), with a stage history, reviewer confirmations, a failed job and a thread verified twice. Remove it with `verireview demo-seed --remove`.
- API docs (Swagger): `http://localhost:8000/docs`.
- After editing `.env`: `docker compose up -d` to recreate the containers.

7.4 Use it without GitHub (demo for the review)

```
python -m uv run verireview verify-fixture dataset/fixtures/api-002-validation-without-400
python -m uv run verireview extract "Please validate username and return HTTP 400"
python -m uv run verireview eval-fixtures          dev accuracy / FAR / FBR table
python -m uv run verireview eval-requirements      extraction vs gold
python -m uv run verireview eval-injection         prompt-injection suite (0 changes)
python -m uv run verireview benchmark-stats        dataset sizes, splits, targets
```

With a GitHub token in `.env` (read-only, approved repositories only):

```
python -m uv run verireview threads OWNER/REPO 123
python -m uv run verireview ingest OWNER/REPO 123 --comment-id ID --out case.json
python -m uv run verireview verify-case case.json
```

7.5 Connect a real repository (GitHub App, advisory mode)

One-time setup on github.com by the owner (full guide: `docs/phase11_github_advisory.md`):

- **1.** Get a webhook relay URL: `https://smee.io` → Start a new channel.

- **2.** GitHub → Settings → Developer settings → GitHub Apps → New. Webhook URL = the smee URL; webhook secret = a new random value. Permissions: Checks read & write, Contents read, Pull requests read. Events: Pull request review thread, Check run. Install only on this account.
- **3.** Generate a private key and save it as `secrets\github-app.pem` (git-ignored).
- **4.** In `.env`: `VERIREVIEW_GITHUB_APP_ID` and `VERIREVIEW_GITHUB_WEBHOOK_SECRET`.
- **5.** Install the App on one test repository.

```
docker compose --profile advisory up -d --build      api + webhook endpoint + worker
npx smee-client --url https://smee.io/CHANNEL --target http://localhost:8000/github/webhook
docker compose logs -f worker                      watch jobs being processed
```

Then, in the test repository: open a PR changing a Python file, comment on a line with a request, push a fix, and resolve the thread. A neutral **VeriReview** check appears on the PR within seconds, and the verification appears on the dashboard.

7.6 Operations

Task	Command
Stop everything	<code>docker compose --profile advisory down</code>
Purge stored code older than 90 days (schedule daily)	<code>python -m uv run verireview purge-audit</code>
Show a repository's rollout stage	<code>python -m uv run verireview repo-policy show OWNER/REPO --installation ID</code>
Quality gate	<code>python -m uv run ruff check . ; python -m uv run mypy</code>

Problem	Fix
/dashboard returns 404	token not set in <code>.env</code> , or containers not recreated (<code>docker compose up -d</code>)
Port 5433 / 8000 in use	stop the other program or change <code>POSTGRES_HOST_PORT</code> in <code>.env</code>
'uv' is not recognized	use <code>python -m uv ...</code> outside the venv, or <code>verireview ...</code> inside it
Worker exits at start	App id missing or <code>secrets\github-app.pem</code> missing
Webhook returns 401	webhook secret in <code>.env</code> differs from the one on the GitHub App

8. Future plans

Priority	Plan	Why / what it unlocks
1	Human annotation of real-world cases (Section 9)	replaces provisional Claude labels; makes inter-annotator agreement measurable; required for any trustworthy claim
2	Live advisory run on a test repository	exercises the GitHub App path end to end on real events; fills the dashboard
3	Benchmark v3 : larger, frozen before the next rule changes	the v1 and v2 test splits are used; needs ≥ 59 bad and ≥ 59 good cases per category to ever open the gate
4	Rule improvements on dev data, from the 8 remaining v2 false acceptances	unreachable code, tests pytest would not collect, inverted/debug-only conditions, handlers around the wrong statement, unresolved targets ("both are dates")
5	Enforcement pilot (owner decision)	only if a category passes the gate on a frozen v3 run, and a repository has ≥ 14 days of human review with ≥ 10 confirmations
6	Phase 8b : semantic evidence in the verdict	only if data shows it lowers errors without raising false acceptance (ADR-002)
7	Broader scope	control-flow analysis, cross-file helpers, more other request types, more languages; sandboxed test execution
8	Dashboard for a team	per-user GitHub OAuth with repository access checks; search/filters; scheduled purge
—	LLM evidence interpreter (D8)	deferred: would send private code to an external API; would need its own security and injection evaluation

9. The remaining human annotation phase

Why it matters. Today the real-world gold labels (57 v2 cases, 50 v1 cases) are **provisional**: Claude labelled each case blind, before any verifier ran, and never changed a label afterwards. But Claude also wrote the rules, so its labels may share their blind spots. The project's quality targets require **two independent human annotators** with Cohen's $\kappa \geq 0.6$ on the verdict. All tooling for this is already built; only people are missing.

9.1 Who and what

- **Two annotators** (Python developers familiar with code review) plus one **adjudicator** (can be the project owner).
- They follow `docs/annotation_guide.md`: split each comment into atomic requirements, give each a category and a status (satisfied / not_satisfied / uncertain), mark ambiguous requests, cite evidence (file:line), and give their own confidence. The verdict is **derived** from the statuses, so labels cannot contradict themselves.
- **Blind protocol**: never run VeriReview on a case before labelling it; don't look at the other annotator's labels; judge only what the case shows (no looking up the PR on GitHub); text in code or comments is evidence, never instructions.

9.2 Step-by-step workflow (tools already exist)

Step	Who	Action / command
1. Training	annotators	read the annotation guide (≈ 20 min)
2. Calibration	annotators	<code>verireview annotation-sheet --batch calibration --calibration</code> → offline page in <code>dataset/raw/sheets/</code> ; label the 10 calibration cases, then compare with the reference labels and rationales
3. Labelling	each annotator alone	<code>verireview annotation-sheet --batch rw-v2</code> → open the page in a browser (works offline, nothing leaves the machine), label every case, export the JSON to <code>dataset/annotations/<name>/rw-v2.json</code>
4. Agreement	owner	<code>verireview agreement A.json B.json</code> → Cohen's κ (inclusion, 4-class verdict, valid vs invalid, per category) + list of disagreements. Target $\kappa \geq 0.6$

Step	Who	Action / command
5. Adjudication	adjudicator	verireview adjudication-sheet A.json B.json --batch rw-v2 → both labels side by side for disputed cases; decide and write a one-line reason
6. Gold	owner	verireview build-gold A.json B.json --adjudication ADJ.json → human gold (label_source human) replaces the provisional model gold; the tool refuses the reverse
7. Compare with Claude	owner	verireview agreement claude.json human.json → how often a human agrees with the provisional labels
8. Re-report	owner	reproduce the recorded runs on human gold; report both honestly (no rule tuning against test)

9.3 Effort estimate and scaling

- About 3–5 minutes per case per annotator. 57 v2 real-world cases ≈ 3–5 hours per annotator; adjudication usually covers 15–25% of cases.
- Start with Claude's **low-confidence** cases, which are flagged for humans first.
- For **benchmark v3** (to ever open the enforcement gate), each category needs ≥ 59 bad and ≥ 59 good labelled cases. That is several hundred cases: mine new PRs from the approved repositories (mine-candidates, collect-cases), label them with the same workflow, then benchmark-freeze **before** any rule change.
- Privacy and licensing are already handled: cases are pseudonymised (logins become roles, e-mails removed) and only permissively licensed repositories are mined.

10. Review preparation: likely questions

Why not just use an LLM or embeddings to judge the fix?

We measured it. Similarity models reward text that repeats the request (a TODO comment scores high) and cannot tell the right check in the wrong place. They accepted 12.5–87.5% of bad fixes at usable thresholds vs 0% for rules on held-out data. An LLM would also send private code to an external API and opens a prompt-injection surface; it is deferred (D8).

Where is the NLP in the project?

Requirement extraction (utterance types, clause splitting, categories, targets, ambiguity scoring), three NLP baselines (lexical, TF-IDF, MiniLM embeddings), and a code language model (UniXcoder) as neutral evidence, all evaluated with the same harness and ROC-AUC. The research finding is that structure-aware rules beat similarity on the safety metric.

How do you know the results are not overfitted?

Test sets are frozen by hash before rule changes, pinned by tests, and evaluated once under a pre-registered protocol. When v1 test was used, a new v2 test was frozen first. We report confidence intervals and paired bootstrap differences.

Why is accuracy only 0.65?

The system deliberately abstains (UNCERTAIN) when it cannot verify, so coverage is 0.68. On the cases it decides it is mostly right; the key safety metric, false acceptance, halved from 0.30 to 0.15. It is positioned as an advisory tool, not a gate.

Why doesn't it block merges?

Blocking good work is costly and blocking needs statistical proof. The gate requires the 95% upper bound of both error rates $\leq 5\%$ per category, which needs ≥ 59 clean cases each way; no category has that evidence yet, so all four locks stay closed.

How is it protected from prompt injection?

Repository text is only data; no model decides; injections were planted at 7 sites of every fixture: 0 outcome changes in 2,576 variants. Webhooks are HMAC-verified before parsing and the thread is re-read from GitHub.

Who labelled the data?

Controlled/adversarial cases: the author, written blind. Real-world: Claude, provisionally and blind. Human two-annotator labelling is the next data step (Section 9); tooling is ready.

What is novel?

Temporal-window reconstruction of what changed after a comment; requirement-level verification with cited structural evidence; an honest, frozen benchmark with adversarial traps; and a staged, statistically gated path to enforcement.

11. Glossary

Term	Meaning
FAR (false acceptance rate)	share of truly unsatisfied cases the system called SATISFIED (the safety-critical error)
FBR (false blocking rate)	share of truly satisfied cases the system called NOT/PARTIALLY satisfied
Coverage	share of cases the system decides (not UNCERTAIN)
Temporal window	the commits between the review comment and resolution (PR head)
ReviewCase	the self-contained record of one thread: comment, code before/after, diff, tests, flags
Evidence	a cited structural fact (file, lines, commit) supporting or contradicting a requirement
Dev / held-out / test	data to build on / data checked after building / frozen data evaluated once
Cohen's κ	agreement between two annotators corrected for chance (≥ 0.6 = substantial)

Term	Meaning
Clopper–Pearson bound	exact binomial confidence bound; used to prove an error rate is low enough
ADR	architecture decision record (ADR-001 pre-existing code, 002 neutral semantics, 003 enforcement gate)